Assignment 1

CSA0903-Java Programming

Name: Sowmiya. D

Reg No.:192511063

Scenario — Campus Smart-Parking System

The university campus has a multi-level parking facility used by students, staff and visitors. The facility has several floors, and each floor has a fixed number of numbered slots. Vehicles entering the campus may be two-wheelers, cars or electric vehicles (EVs). EVs may also use charging slots, which are charged separately.

When a vehicle enters, the system must allocate a suitable free slot, issue a parking ticket with a unique ticket number, and record the entry time. When the vehicle leaves, the system must release the slot and compute the parking fee. Fees depend on the vehicle type and the duration of parking. Staff and students holding a valid monthly pass park free of charge, while visitors pay the full fee.

The facility may contain:

1 floor × 20 slots → 3 floors × 50 slots → 5 floors × 100 slots

You are required to analyse the problem, design an object-oriented model for it, implement the model in Java and validate it. Your design must remain correct and easy to extend as the facility and its rules grow.

Design Challenge:

The university now wants to add new vehicle categories and a new pricing rule without rewriting the classes that already work.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 9 – Introduce Abstraction 4 Marks

Bloom's Level: K4 – Analyse

Use abstract classes and interfaces to separate what the system does from how it is done.

Discuss:

- Which class should be abstract (for example: Vehicle) and which methods should be abstract
- Which capability should be an interface (for example: Chargeable for EVs, Payable for fee-paying users)
- Why you chose an abstract class in one place and an interface in another
- How a class can obtain multiple behaviours through interfaces
- Where a default or static method in an interface is useful

Alternative Concept and Complexity Analysis

Vehicle is an abstract class and Chargeable is an interface. If Vehicle were changed to an interface with default methods, and Chargeable to an abstract class, what would the solution be? Identify what can no longer be expressed (constructors, state, multiple inheritance) and analyse any difference in time per method call and space per object.

Evidence Expected

A justified use of at least one abstract class and one interface, with the reasoning behind each choice.

Solution:

TASK 9 – Introduce Abstraction

Campus Smart-Parking System – 4 Marks

This solution continues the same vehicle hierarchy used in Tasks 6, 7 and 8:

- Vehicle – Abstract class
- TwoWheeler – Subclass of Vehicle
- Car – Subclass of Vehicle
- ElectricCar – Subclass of Car
- Chargeable – Interface implemented by ElectricCar

1. Introduction to Abstraction

Abstraction is an object-oriented programming concept that hides implementation details and displays only essential features.

Java achieves abstraction using:

1. Abstract classes.
2. Interfaces.

In the Campus Smart-Parking System, abstraction separates common vehicle behaviour from vehicle-specific implementations.

2. Abstract Class – Vehicle

The Vehicle class is declared abstract because it represents a general vehicle and should not be instantiated directly.

It contains:

- regNo – Stores the vehicle registration number.
- getRegNo() – Returns the registration number.
- calculateFee() – Calculates the parking fee.

- ratePerHour() – An abstract method implemented by subclasses.

The abstract method ratePerHour() forces every concrete vehicle subclass to provide its own hourly parking rate.

Example:

```
abstract class Vehicle {
   private final String regNo;
   Vehicle(String regNo) {
      this.regNo = regNo;
   }\
   public String getRegNo() {
      return regNo;
   }
   protected abstract double ratePerHour();


   public double calculateFee(int minutes) {
      int hours = (minutes + 59) / 60;
      return hours * ratePerHour();
   }
}
```

3. Interface – Chargeable

The Chargeable interface represents the charging capability of electric vehicles.

It declares the chargingFee() method, which must be implemented by classes that support electric charging.

```
interface Chargeable {
   double chargingFee(int kWh);
}
```

The ElectricCar class implements this interface because it supports battery charging.

4. Why Abstract Class and Interface Are Used

Abstract Class	**Interface**

Represents common vehicle identity.	Represents charging capability.
Stores registration number.	Defines charging behaviour.
Supports constructors and instance variables.	Does not have ordinary instance variables or constructors.
Supports shared method implementation.	Defines a contract for implementing classes.

An abstract class is suitable for Vehicle because all vehicles share common data and behaviour.

An interface is suitable for Chargeable because charging is an additional capability that only certain vehicles support.

5. Multiple Behaviours Through Interfaces

Java allows a class to implement multiple interfaces.

For example, an ElectricCar can implement both Chargeable and another interface such as Insurable.

```
class ElectricCar extends Car
    implements Chargeable, Insurable {
}
```

This allows ElectricCar to inherit vehicle features from Car and obtain additional behaviours through interfaces.

6. Default and Static Methods in Interfaces

Default method: A default method provides a common implementation that implementing classes can use or override.

Example:

```
default void chargingStatus() {
   System.out.println("Charging supported");
}
```

Static method: A static method belongs to the interface and can be called using its interface name.

Example:

```
static double standardChargingRate() {
   return 12.0;
}
```

The default method is useful for displaying a common charging status, while the static method provides a standard charging rate.

7. Alternative Concept – Swapping Abstract Class and Interface

If Vehicle were changed into an interface with default methods and Chargeable into an abstract class, the design would become less suitable.

Factor	**Original Design**	**Alternative Design**
Vehicle	Abstract class	Interface
Chargeable	Interface	Abstract class
Constructors	Supported in Vehicle	Not supported in Vehicle interface
Instance variables	Supported	Not supported in Vehicle interface
Multiple inheritance	Multiple interfaces supported	Cannot extend two abstract classes
Shared state	Stored in Vehicle	Must be moved elsewhere
Flexibility	Supports different capabilities	More restrictive

What can no longer be expressed?

- A Vehicle interface cannot have constructors.
- It cannot store ordinary instance variables such as regNo.
- A class cannot extend both Car and a Chargeable abstract class because Java supports only single class inheritance.
- Multiple behaviours can still be obtained through interfaces.

Time Complexity Analysis

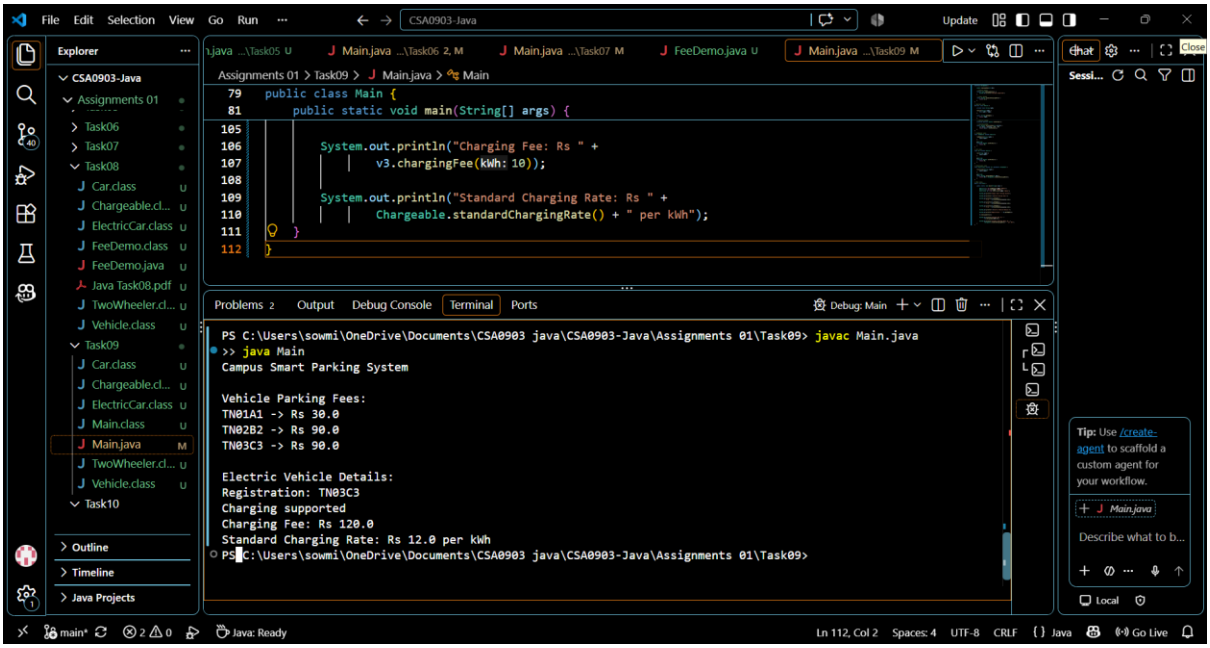
Both abstract class methods and interface methods generally have O(1) time complexity for ordinary method dispatch.

There is no significant inherent time-complexity difference between the two designs. Actual performance depends on JVM optimisation and implementation.

Space Complexity Analysis

- Original design: Each vehicle object stores its registration number and other instance fields.
- Alternative design: Registration information must be stored in implementing classes or another object.
- Interfaces do not add separate per-object instance storage.
- An abstract class can contain instance variables, which contribute to object storage.

Conclusion: The original design is more suitable because Vehicle represents a common identity with shared state, whereas Chargeable represents an optional capability. This design improves reusability, flexibility and maintainability.


File Edit Selection View Go Run
CSA0903-Java
Update
Explorer
CSA0903-Java
Assignments 01
Task06
Task07
Task08
Car.class
Chargeable.cl...
ElectricCar.class
FeeDemo.class
FeeDemo.java
Java Task08.pdf
TwoWheeler.cl...
Vehicle.class
Task09
Car.class
Chargeable.cl...
ElectricCar.class
Main.class
Main.java
TwoWheeler.cl...
Vehicle.class
Task10
Outline
Timeline
Java Projects
Main.java ...\Task05
Main.java ...\Task06 2, M
Main.java ...\Task07 M
FeeDemo.java
Main.java ...\Task09 M
Assignments 01 > Task09 > Main.java > Main
79 public class Main {
81 public static void main(String[] args) {
105 System.out.println("Charging Fee: Rs " +
106 v3.chargingFee(kWh: 10));
107
108 System.out.println("Standard Charging Rate: Rs " +
109 Chargeable.standardChargingRate() + " per kWh");
110 }
111 }
Problems 2
Output
Debug Console
Terminal
Ports
Debug: Main
PS C:\Users\sowmi\OneDrive\Documents\CSA0903 java\CSA0903-Java\Assignments 01\Task09> javac Main.java
>> java Main
Campus Smart Parking System
Vehicle Parking Fees:
TN01A1 -> Rs 30.0
TN02B2 -> Rs 90.0
TN03C3 -> Rs 90.0
Electric Vehicle Details:
Registration: TN03C3
Charging supported
Charging Fee: Rs 120.0
Standard Charging Rate: Rs 12.0 per kWh
PS C:\Users\sowmi\OneDrive\Documents\CSA0903 java\CSA0903-Java\Assignments 01\Task09>
chat
Sessi...
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Main.java
Describe what to b...
Local
main*
2 0
Java: Ready
Ln 112, Col 2
Spaces: 4
UTF-8
CRLF
Java
Go Live